16.1.1.2 Splitting entities

The `Post` entity can be split into several sub-entities according to the three distinct set of attributes:

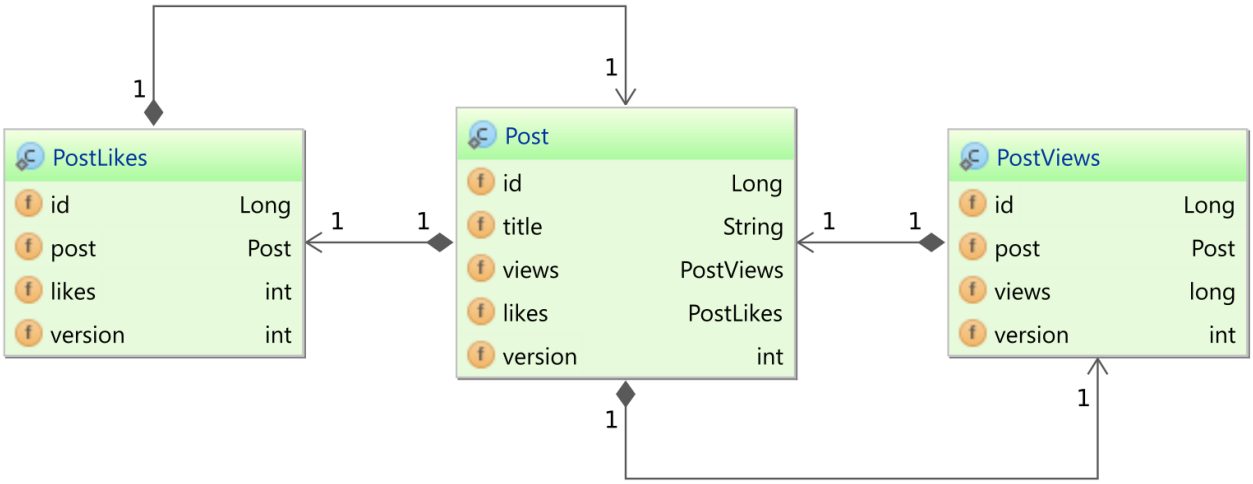

Figure 16.3: Post entity version split

While the `title` attribute remains in the `Post` parent entity, the `likes`, and the `views` attributes are moved to distinct entities. The `PostLikes` and `PostViews` entities are associated with the `Post` parent entity in a bidirectional one-to-one relationship. The `PostLikes` and `PostViews` entity identifiers are also foreign keys to the `post` table primary key.

Each entity has its own `version` attribute. Whenever the `Post` `title` is changed, it is only the `Post` entity `version` that is checked and incremented. When the `views` attribute is updated, only the `PostViews` entity is going to be affected. The same is true for incrementing `likes` which are stored in the `PostLikes` entity.

While breaking a larger entity into several sub-entities can help address optimistic locking conflicts, this strategy has its price. This rather extreme data normalization strategy can have an impact on read operation performance because data is scattered across several tables. If the whole aggregate is needed to be fetched, the data access layer will require to join several tables or execute additional secondary select statements.

The second-level cache can mitigate the read operation performance penalty. Actually, the root entity split can improve the second-level cache performance, especially for read-through strategies (e.g. `NONSTRICT_READ_WRITE`). If the `views` attribute is modified, only the `PostViews` cache entry needs to be invalidated, whereas the `Post` and the `PostLikes` remain unaffected.

When running the previous exercise, there is no longer any conflict being generated:

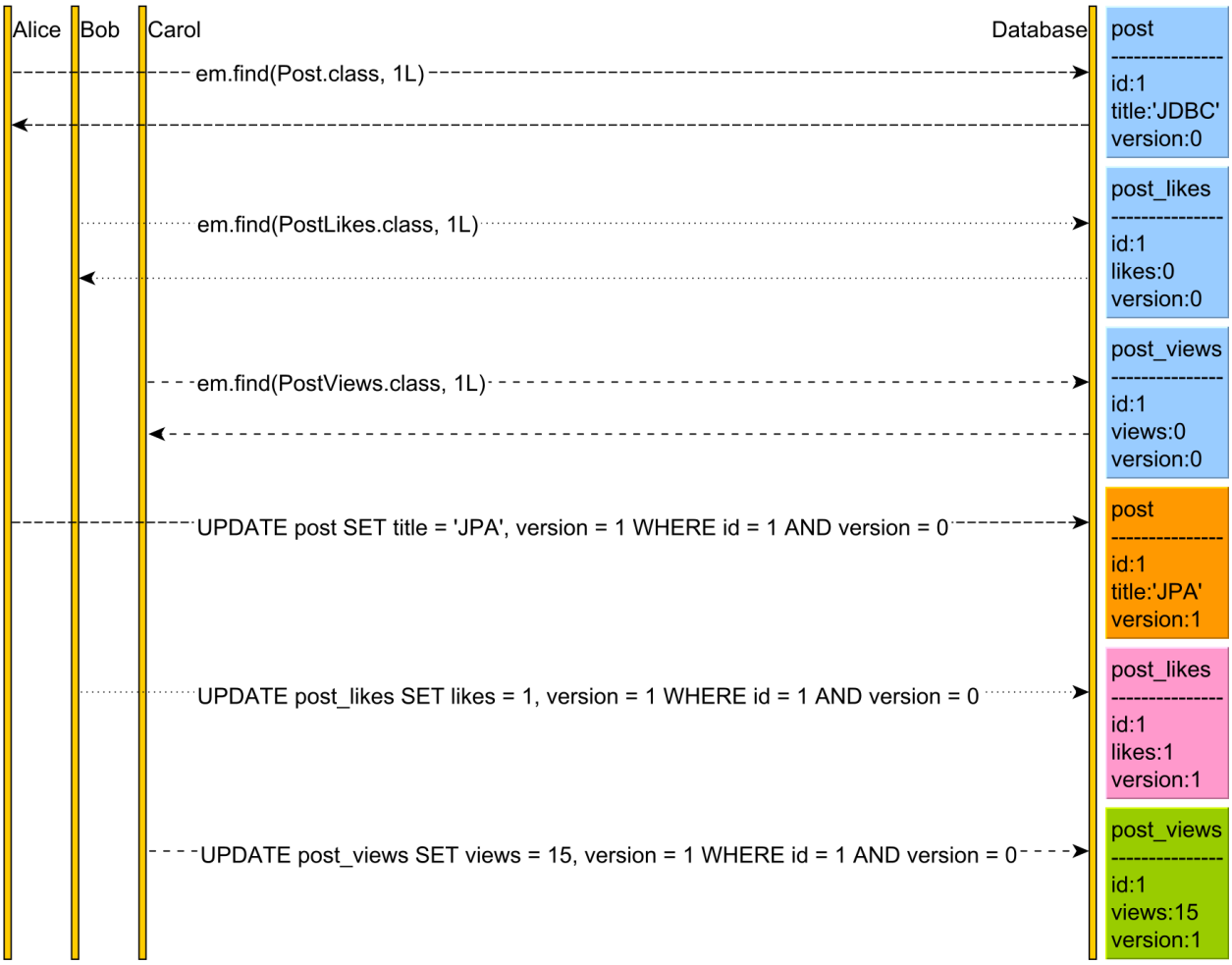

Figure 16.4: Optimistic locking multiple versions

Lost updates are prevented at the entity-level only so the three distinct attributes can be updated concurrently without generating any conflict. However, conflicts can still occur if the same attribute is getting updated by two concurrent transactions.

Designing a Domain Model must take into consideration both read and write data access patterns. Splitting entities by write responsibility can reduce optimistic locking false positives when the write ratio is relatively high.

16.1.1.3 Versionless optimistic locking

Although having a numerical version attribute is the most common optimistic concurrency control strategy, Hibernate offers a versionless optimistic locking mechanism which is not supported by JPA 2.1 specification. To switch to the versionless optimistic locking mechanism, the `@OptimisticLocking` annotation must be configured at the entity level.

The `org.hibernate.annotations.OptimisticLocking` annotation comes with a `type` attribute that can take the following values:

- NONE - The implicit optimistic locking mechanism is disabled.
- VERSION - The implicit optimistic locking mechanism uses a numerical or a timestamp version attribute.
- DIRTY - The implicit optimistic locking mechanism uses only the attributes that have been modified in the currently running Persistence Context.
- ALL - The implicit optimistic locking mechanism uses all entity attributes.

`OptimisticLockType.ALL` and `OptimisticLockType.DIRTY` also require the `@DynamicUpdate` annotation because update statements must be rewritten so that either all attributes or the ones that were modified are included in the where clause criteria. For these two optimistic lock types, the entity versioning mechanism is based on the hydrated state snapshot that was stored when the entity was first loaded in the currently running Persistence Context.

To see how the `OptimisticLockType.ALL` option works, consider the following `Post` entity mapping:

```
@Entity @Table(name = "post") @DynamicUpdate
@OptimisticLocking(type = OptimisticLockType.ALL)
public class Post {

    @Id
    private Long id;

    private String title;

    //Getters and setters omitted for brevity
}
```

When using the aforementioned `Post` entity to rerun the example defined at the beginning of the Hibernate implicit optimistic locking section, Hibernate generates the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

UPDATE post SET title = 'High-Performance JDBC'
WHERE id = 1 AND title = 'High-Performance Java Persistence'
```

If the `Post` entity had more attributes, all of them would be included in the SQL where clause.

> The `OptimisticLockType.ALL` is useful when the underlying database table cannot be altered in order to add a numerical `version` column. Because it takes into consideration all entity attributes, the `OptimisticLockType.ALL` option behaves just like a single global version attribute, and write conflicts can occur even if two concurrent transactions are modifying non-overlapping attribute sets.

Even if the entity splitting method can address the non-overlapping attribute sets conflict, too much data normalization can affect read operation performance. The `OptimisticLockType.DIRTY` option can deal with this issue, and so lost updates are prevented for the currently modified attributes.

To demonstrate it, the following `Post` entity mapping is going to be used while running the same test case employed in the resolve optimistic locking conflicts section:

```
@Entity @Table(name = "post") @DynamicUpdate
@OptimisticLocking(type = OptimisticLockType.DIRTY)
public class Post {

    @Id
    private Long id;

    private String title;

    private long views;

    private int likes;

    //Getters and setters omitted for brevity
}
```

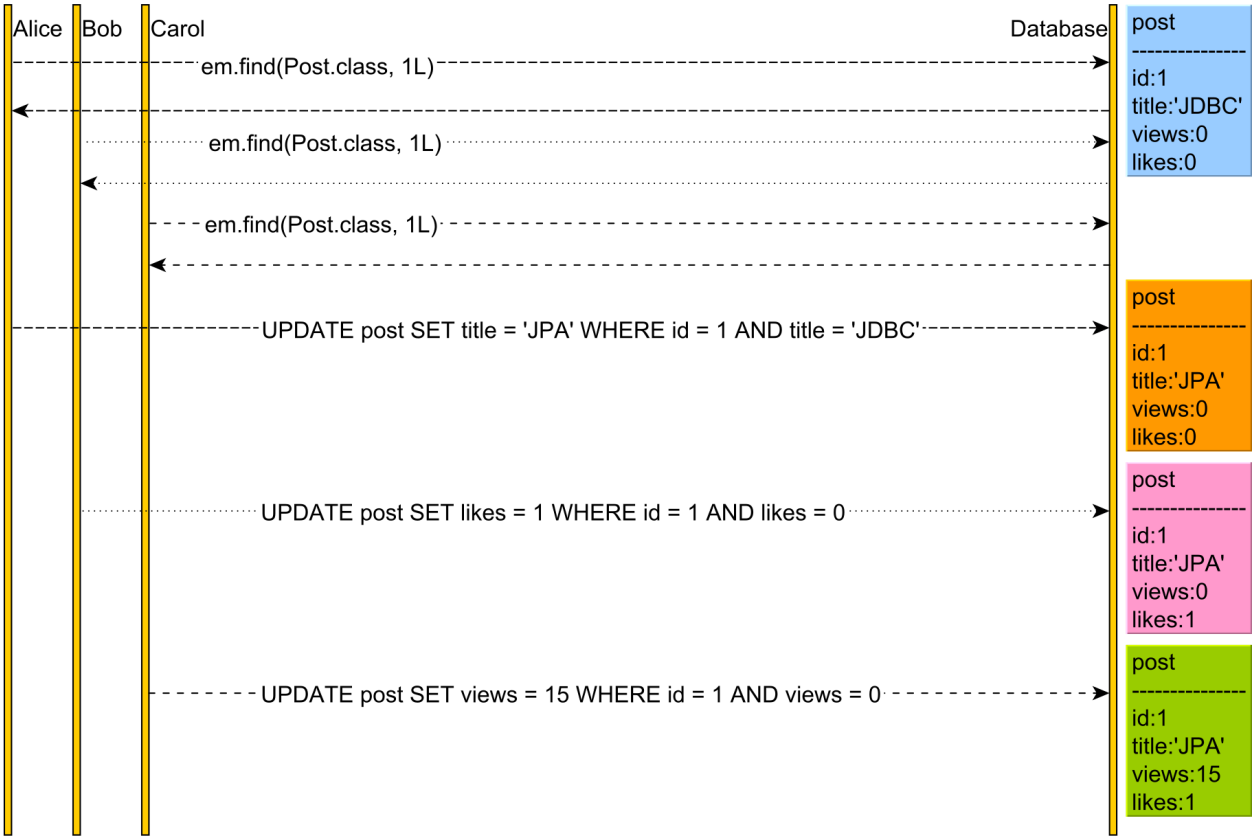

Figure 16.5: Optimistic locking dirty attributes

The `OptimisticLockType.DIRTY` option allows concurrent users to update distinct attributes without causing any conflict. However, conflicts can still occur when two concurrent transactions are updating the same attribute. Therefore, lost updates are prevented on a per-attribute basis.

For heavy-write data access layers, it is not uncommon to split an entity into multiple parts, each individual subentity containing attributes that need to be updated atomically. If, from a writing perspective, attributes are independent, then the `OptimisticLockType.DIRTY` mechanism is also a viable alternative.

Preventing lost updates is essential for data integrity, but the prospect of having transactions aborted due to non-overlapping attribute changes is undesirable. To cope with this issue, entities need to be carefully modeled based on both read and write data access patterns.

16.1.1.3.1 OptimisticLockType.DIRTY update caveat

In spite of being very useful for preventing optimistic locking conflicts, the `OptimisticLockType.DIRTY` mechanism has one limitation: it does not work with the `Session.update()` method.

```
Post detachedPost = doInJPA(entityManager -> {
    LOGGER.info("Alice loads the Post entity");
    return entityManager.find(Post.class, 1L);
});

executeSync(() -> {
    doInJPA(entityManager -> {
        LOGGER.info("Bob loads the Post entity and modifies it");
        Post post = entityManager.find(Post.class, 1L);
        post.setTitle("Hibernate");
    });
});

doInJPA(entityManager -> {
    LOGGER.info("Alice updates the Post entity");
    detachedPost.setTitle("JPA");
    entityManager.unwrap(Session.class).update(detachedPost);
});
```

When running the test case above, Hibernate generates the following statements:

```
-- Alice loads the Post entity
SELECT p.id AS id1_0_0_, p.likes AS likes2_0_0_, p.title AS title3_0_0_,
       p.views AS views4_0_0_
FROM   post p
WHERE  p.id = 1

-- Bob loads the Post entity and modifies it
SELECT p.id AS id1_0_0_, p.likes AS likes2_0_0_, p.title AS title3_0_0_,
       p.views AS views4_0_0_
FROM   post p
WHERE  p.id = 1

UPDATE post SET title = 'Hibernate' WHERE id = 1 AND title = 'JDBC'

-- Alice updates the Post entity
UPDATE post SET likes=0, title='JPA', views=0 WHERE id=1
```

Bob's update benefits from dirty attribute optimistic locking, just as expected. On the other hand, Alice's update is not using any optimistic locking at all.

That is because the reattached `Post` entity misses the loaded state information, so the dirty checking mechanism cannot be executed in this case. For this reason, Hibernate schedules an update statement that simply copies the current entity state to the underlying database record. Unfortunately, this can lead to lost updates since Alice is not aware of Bob's latest modification. If optimistic locking were working, Alice's update would be prevented.

The `@SelectBeforeUpdate` annotation allows Hibernate to fetch the entity snapshot prior to executing the update query. This way, Hibernate can run the dirty checking mechanism and make sure that the update is really necessary.

```
@Entity(name = "Post") @Table(name = "post")
@OptimisticLocking(type = OptimisticLockType.DIRTY)
@DynamicUpdate
@SelectBeforeUpdate
public class Post {

    @Id
    private Long id;

    private String title;

    private long views;

    private int likes;

    //Getters and setters omitted for brevity
}
```

Unfortunately, even when using `@SelectBeforeUpdate`, the optimistic locking mechanism is still circumvented, and Alice update transaction executes the following statements:

```
SELECT p.id AS id1_0_0_, p.likes AS likes2_0_0_, p.title AS title3_0_0_,
       p.views AS views4_0_0_
FROM   post p
WHERE  p.id = 1

UPDATE post SET title='JPA' WHERE id = 1
```

The dynamic update works since the update statement contains only the modified attribute, but there is no optimistic locking filtering criteria.

If Alice uses the `EntityManager.merge()` operation:

```
doInJPA(entityManager -> {
    detachedPost.setTitle("JPA");
    entityManager.merge(detachedPost);
});
```

Hibernate executes the following SQL statements:

```
SELECT p.id AS id1_0_0_, p.likes AS likes2_0_0_, p.title AS title3_0_0_,
       p.views AS views4_0_0_
FROM   post p
WHERE  p.id = 1

UPDATE post SET title='JPA' WHERE id = 1 AND title = 'Hibernate'
```

The optimistic locking mechanism is used, but it is relative to the newly loaded entity state. This time, lost updates can only be detected if, while merging the detached entity, Carol would update the same `Post` entity `title`.

Statefulness to the rescue

Unfortunately, Bob's update is still undetected by the versionless optimistic locking mechanism. The current entity state alone is no longer sufficient when merging a detached entity version because the Persistence Context cannot determine which attributes have been changed and which reference values are to be used in the where clause filtering criteria.

To fix it, the entity must store the loading-time attribute state so that the Persistence Context can use it later for the optimistic locking where clause criteria, therefore, preventing any lost update occurrence. However, this is impractical, and so the entity state must be stored either in a stateful Persistence Context, or its loading-time version value be saved separately.

Using a numerical version is practical, but it can lead to optimistic locking conflicts because all attributes are treating as a global all-or-nothing update attribute set. On the other hand, the Persistence Context does not need to be closed. Only the database connection needs to be released to allow other concurrent transactions to execute in the user think time. The Persistence Context can be kept open so that entities never become detached. This way, the entity loading-time state is never lost, and the versionless optimistic locking mechanism will work even across multiple transactions. When using Java EE, a `PersistenceContextType.EXTENDED` can be used inside a `@Stateful` EJB. Spring Webflow allows registering a Persistence Context in the `HttpSession` so that the `EntityManager` remains open throughout the whole lifecycle of the current flow.

16.2 The explicit locking mechanism

While the implicit locking mechanism is suitable for many application concurrency control requirements, there might be times when a finer-grained locking strategy is needed. JPA offers a concurrency control API, on top of which the application developer can implement really complex data integrity rules. The explicit locking mechanism works for both pessimistic and optimistic locking.

For pessimistic concurrency control, JPA abstracts the database-specific locking semantics, and, depending on the underlying database capabilities, the application developer can acquire exclusive or shared locks.

If the implicit optimistic locking mechanism controls the entity version automatically, and the application developer is not allowed to make changes to the underlying version attribute, the explicit optimistic lock modes allow incrementing an entity version even if the entity was not changed by the currently running transaction. This is useful when two distinct entities need to be correlated so that a child entity modification can trigger a parent entity version incrementation.

JPA offers various `LockModeType(s)` that can be acquired for the direct loading mechanism (e.g. `entityManager.find`, `entityManager.lock`, `entityManager.refresh`) as well as for any JPQL or Criteria API query (e.g. `Query.setLockMode()`).

The following table lists all `LockModeType(s)` that can be acquired by a particular entity:

Table 16.1: `LockModeType(s)`

Lock Mode Type	Description
NONE	In the absence of explicit locking, the application uses the default implicit locking mechanism.
OPTIMISTIC or READ	Issues a version check upon transaction commit.
OPTIMISTIC_FORCE_INCREMENT or WRITE	Increases the entity version prior to committing the current running transaction.
PESSIMISTIC_FORCE_INCREMENT	An exclusive database lock is acquired, and the entity version is incremented right away.
PESSIMISTIC_READ	A shared database lock is acquired to prevent any other transaction from acquiring an exclusive lock.
PESSIMISTIC_WRITE	An exclusive lock is acquired to prevent any other transaction from acquiring a shared/exclusive lock.

The following sections will analyze each individual `LockModeType` in greater detail.

16.2.1 PESSIMISTIC_READ and PESSIMISTIC_WRITE

To acquire row-level locks, JPA defines two `LockModeType`: `PESSIMISTIC_READ`, for shared locks, and `PESSIMISTIC_WRITE`, for exclusive locks. Unfortunately, there is no standard definition for acquiring shared and exclusive locks, and each database system defines its own syntax.

Oracle

Only exclusive locks are supported for which Oracle defines the `FOR UPDATE`[a] clause. Rows that were selected with the `FOR UPDATE` clause cannot be locked or modified until the current transaction either commits or rolls back.

[a] https://docs.oracle.com/database/121/SQLRF/statements_10002.htm#SQLRF01702

SQL Server

SQL Server does not define a `FOR UPDATE` select statement clause, but instead it defines several table hints[a]. The `WITH (HOLDLOCK, ROWLOCK)` is equivalent to acquiring a shared lock until the current running transaction is ended, whereas the `WITH (UPDLOCK, HOLDLOCK, ROWLOCK)` hint can be used to acquire an exclusive lock.

[a] https://msdn.microsoft.com/en-us/library/ms187373.aspx

PostgreSQL

The select clause can take multiple locking clauses[a] among which `FOR SHARE` is used to acquire a shared lock, whereas `FOR UPDATE` takes an exclusive lock on each selected row.

[a] https://www.postgresql.org/docs/current/static/sql-select.html#SQL-FOR-UPDATE-SHARE

MySQL

Just like PostgreSQL, the `FOR UPDATE` clause can be used to acquire an exclusive lock, while LOCK IN SHARE MODE[a] is used for shared locks.

[a] http://dev.mysql.com/doc/refman/5.7/en/innodb-locking-reads.html

When using Hibernate, the application developer needs not to worry about the locking syntax employed by the underlying database system. To acquire an exclusive lock, the `PESSIMISTIC_WRITE` lock type must be used, and Hibernate will pick the underlying `Dialect` lock clause.

For instance, when running the following entity lock acquisition request on PostgreSQL:

```
Post post = entityManager.find(Post.class, 1L, LockModeType.PESSIMISTIC_WRITE);
```

Hibernate is going to generate the following query:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1
FOR UPDATE
```

If the relational database offers support for acquiring shared locks explicitly, the `PESSIMISTIC_READ` lock type must be used instead. When fetching an entity directly using the `PESSIMISTIC_READ` lock type on PostgreSQL:

```
Post post = entityManager.find(Post.class, 1L, LockModeType.PESSIMISTIC_READ);
```

Hibernate is going to use the `FOR SHARE` select clause:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1
FOR SHARE
```

If the underlying database does not support shared locks, when using the `PESSIMISTIC_READ` lock type, an exclusive lock is acquired instead. When running the previous `PESSIMISTIC_READ` direct fetching example on Oracle, Hibernate will use a `FOR UPDATE` select clause.

Although it is much more convenient to lock entities at the moment they are fetched from the database, entities can also be locked even after they are loaded in the currently running Persistence context.

For this purpose, the `EntityManager` interface defines the `lock` method which takes a managed entity and a `LockModeType`:

```
Post post = entityManager.find(Post.class, 1L);
entityManager.lock(post, LockModeType.PESSIMISTIC_WRITE);
```

When running the aforementioned example, Hibernate is going to execute the following statements:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

SELECT id
FROM   post
WHERE  id = 1 AND version = 0
FOR UPDATE
```

Only a managed entity can be passed to the `lock` method when using the Java Persistence API. Otherwise, an `IllegalArgumentException` is being thrown indicating that the entity is not contained within the currently running Persistence Context.

On the other hand, the Hibernate native API offers entity reattachment upon locking as demonstrated by the following example:

```
Post post = doInJPA(entityManager -> {
    return entityManager.find(Post.class, 1L);
});

doInJPA(entityManager -> {
    LOGGER.info("Lock and reattach");
    Session session = entityManager.unwrap(Session.class);
    session.buildLockRequest(
        new LockOptions(LockMode.PESSIMISTIC_WRITE))
    .lock(post);
    post.setTitle("High-Performance Hibernate");
});
```

When running the test case above, Hibernate manages to acquire an exclusive lock on the associated database record while also propagating the entity state change to the database:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

-- Lock and reattach
SELECT id
FROM   post
WHERE  id = 1 AND version = 0
FOR UPDATE

UPDATE post
SET body = 'Chapter 17 summary', title = High-Performance Hibernate'
WHERE id = 1 AND version = 0
```

Because the detached entity becomes managed, the entity modification triggers an update statement at flush time.

16.2.1.1 Lock scope

By default, the lock scope is bound to the entity that is being locked explicitly. However, just like other entity state transitions, the lock acquisition request can be cascaded to child associations like the `PostDetails` and `PostComment(s)` entities in the next diagram.

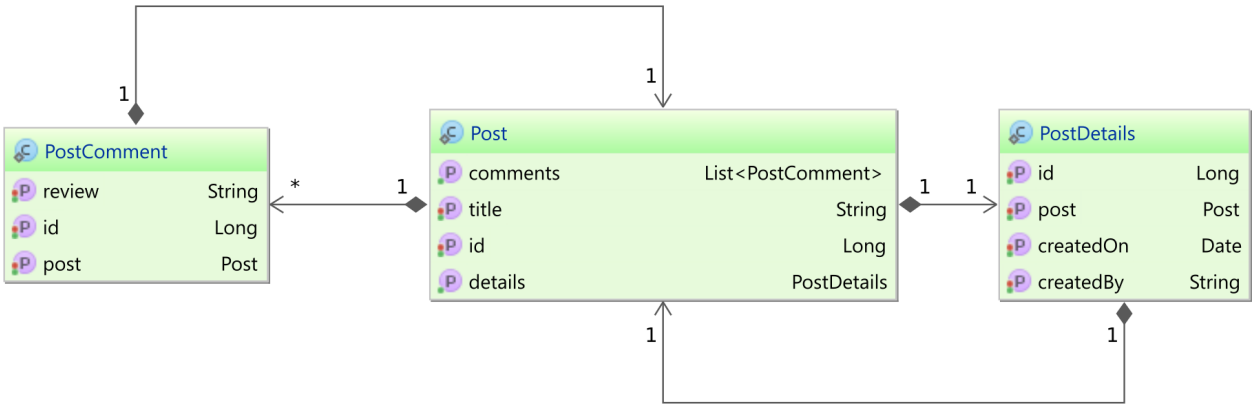

Figure 16.6: Post, PostDetails, and PostComment

The easiest way to lock a whole entity graph is to apply the `LockModeType` at the entity query level.

When executing the following entity query:

```
Post post = entityManager.createQuery(
    "select p " +
    "from Post p " +
    "join fetch p.details " +
    "join fetch p.comments " +
    "where p.id = :id", Post.class)
.setParameter("id", 1L)
.setLockMode(LockModeType.PESSIMISTIC_WRITE)
.getSingleResult();
```

Hibernate generates the following SQL query:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_, pd.id AS id1_2_1_,
       pd.created_by AS created_2_2_1_, pd.created_on AS created_3_2_1_,
       pd.version AS version4_2_1_, pc.id AS id1_1_2_,
       pc.post_id AS post_id4_1_2_, pc.review AS review2_1_2_,
       pc.version AS version3_1_2_, pc.post_id AS post_id4_1_0__,
       pc.id AS id1_1_0__
FROM   post p
INNER JOIN post_details pd ON p.id = pd.id
INNER JOIN post_comment pc ON p.id = pc.post_id
WHERE  p.id = 1
FOR UPDATE
```

The `FOR UPDATE` clause is applied to all records that are being selected, therefore, the whole result set is being locked. In this particular case, the lock scope depends on the query filtering criteria.

Aside from entity queries, Hibernate can also propagate a lock acquisition request from a parent entity to its children when using direct fetching. For this purpose, the child associations must be annotated with the Hibernate specific `CascadeType.Lock`[1] attribute.

`CascadeType.Lock` can also be inherited implicitly when the child association is annotated with the `CascadeType.ALL` attribute.

[1] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/annotations/CascadeType.html#LOCK

To demonstrate how the lock can be cascaded, the `Post` entity is changed so that the `CascadeType.ALL` attribute is set on both `comments` and `details` child associations:

```
@OneToMany(cascade = CascadeType.ALL, mappedBy = "post", orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();

@OneToOne(cascade = CascadeType.ALL, mappedBy = "post", orphanRemoval = true,
          fetch = FetchType.LAZY, optional = false)
private PostDetails details;
```

The implicit or explicit `CascadeType.Lock` is not sufficient because the `LockRequest`[2] declares a `scope` attribute which is disabled by default. For the lock to be cascaded, the `scope` attribute must be set to true as in the following example:

```
Post post = entityManager.find(Post.class, 1L);
entityManager.unwrap(Session.class)
.buildLockRequest(
    new LockOptions(LockMode.PESSIMISTIC_WRITE))
.setScope(true)
.lock(post);
```

However, when executing the test case above, Hibernate is going to lock only the `Post` entity:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

SELECT id
FROM   post
WHERE  id = 1 AND version = 0
FOR UPDATE
```

For managed entities, Hibernate does not cascade the lock acquisition request even if the scope attribute is provided, therefore, the entity query alternative is preferred.

When locking a detached entity graph, Hibernate is going to reattach every entity that enabled cascade propagation while also propagating the lock request.

[2] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/Session.LockRequest.html

```
Post post = doInJPA(entityManager -> {
    return entityManager.createQuery(
        "select p " +
        "from Post p " +
        "join fetch p.details " +
        "join fetch p.comments " +
        "where p.id = :id", Post.class)
    .setParameter("id", 1L)
    .getSingleResult();
});

doInJPA(entityManager -> {
    entityManager.unwrap(Session.class)
    .buildLockRequest(
        new LockOptions(LockMode.PESSIMISTIC_WRITE))
    .setScope(true)
    .lock(post);
});
```

When executing the test case above, Hibernate generates the following queries:

```
SELECT p.id AS id1_0_0_, pd.id AS id1_2_1_, pc.id AS id1_1_2_,
       p.body AS body2_0_0_, p.title AS title3_0_0_, p.version AS version4_0_0_,
       pd.created_by AS created_2_2_1_, pd.created_on AS created_3_2_1_,
       pd.version AS version4_2_1_, pc.post_id AS post_id4_1_2_,
       pc.review AS review2_1_2_, pc.version AS version3_1_2_,
       pc.post_id AS post_id4_1_0__, pc.id AS id1_1_0__
FROM   post p
INNER JOIN post_details pd ON p.id = pd.id
INNER JOIN post_comment pc ON p.id = pc.post_id
WHERE  p.id = 1

SELECT id FROM post_comment WHERE id = 2 AND version = 0 FOR UPDATE
SELECT id FROM post_comment WHERE id = 3 AND version = 0 FOR UPDATE

SELECT id FROM post_details WHERE id = 1 AND version = 0 FOR UPDATE

SELECT id FROM post WHERE id =1 AND version =0 FOR UPDATE
```

Not only the `Post` entity is being locked but also the `PostDetails` and every `PostComment` child entity.

However, if the `Post` entity is loaded without initializing any child association:

```
Post post = doInJPA(entityManager -> (Post) entityManager.find(Post.class, 1L));
```

When running the previous test case, Hibernate is going to execute the following statements:

```
SELECT p.id AS id1_2_0_, p.created_by AS created_2_2_0_,
       p.created_on AS created_3_2_0_, p.version AS version4_2_0_
FROM   post_details p
WHERE  p.id = 1

SELECT id from post_details WHERE id =1 AND version = 0 FOR UPDATE

SELECT id from post WHERE id =1 AND version = 0 FOR UPDATE
```

Only the `Post` and `PostDetails` entities are locked this time. Because the `PostDetails` entity had not been fetched previously, the detached `Post` entity was using a proxy which only held the child association identifier and the child entity type. The one-to-one `PostDetails` association propagates all entity state transitions. Hence, the lock acquisition request is going to be applied to the `PostDetails` proxy as well. When being reassociated, the `@OneToOne` and `@ManyToOne` associations are fetched right away, and the lock is, therefore, propagated.

On the other hand, the `PostComment` child entries are not locked because Hibernate needs not to fetch `@OneToMany` and `@ManyToMany` associations upon reattaching the parent `Post` entity. The lock acquisition request is cascaded to child collections only if the collection is already initialized.

Locking too much data can hurt scalability because, once a row-level lock is acquired, other concurrent transactions that need to modify this record are going to be blocked until the first transaction either commits or rolls back. The lock cascading works only with detached entities and only if the `@OneToMany` and `@ManyToMany` associations have been previously fetched.

Being applicable to both managed and detached entities and giving better control over what entities are getting locked, the entity query locking mechanism is a much better alternative than entity lock event cascading.

16.2.1.2 Lock timeout

When acquiring a row-level lock, it is good practice to set a timeout value for which the current request is willing to wait before giving up. Depending on the current database `Dialect`, if the timeout value is greater than zero, Hibernate can use it to limit the lock acquisition request interval.

For the Hibernate native API, the timeout value can be supplied like this:

```
entityManager.unwrap(Session.class)
.buildLockRequest(
    new LockOptions(LockMode.PESSIMISTIC_WRITE)
        .setTimeOut((int) TimeUnit.SECONDS.toMillis(3))
    )
.lock(post);
```

With JPA, the timeout value is given through the following hint:

```
entityManager.lock(post, LockModeType.PESSIMISTIC_WRITE,
    Collections.singletonMap(
        "javax.persistence.lock.timeout",
        TimeUnit.SECONDS.toMillis(3)
    )
);
```

When running the aforementioned lock acquisition request on Oracle, Hibernate generates the following SQL query:

```
SELECT id
FROM   post
WHERE  id = 1 AND version = 0
FOR UPDATE WAIT 3
```

Even if the timeout value is given in milliseconds, the Hibernate `Dialect` converts it to the underlying database supported format (e.g. seconds for Oracle).

To avoid any waiting, Hibernate comes with a `NO_WAIT` lock option which simply sets the timeout value to 0.

```
entityManager.unwrap(Session.class)
.buildLockRequest(
    new LockOptions(LockMode.PESSIMISTIC_WRITE)
    .setTimeOut(LockOptions.NO_WAIT))
.lock(post);
```

The JPA alternative looks as follows:

```
entityManager.lock(post, LockModeType.PESSIMISTIC_WRITE,
    Collections.singletonMap("javax.persistence.lock.timeout", 0)
);
```

When running the lock acquisition request above on PostgreSQL, Hibernate is going to use the `NO WAIT` PostgreSQL clause:

```
SELECT id
FROM   post
WHERE  id = 1 AND version = 0
FOR UPDATE NOWAIT
```

The `LockOptions.NO_WAIT` option can only be used only if the underlying database supports such a construct (e.g. Oracle and PostgreSQL). For other database systems, this option is ignored and a regular pessimistic write lock clause is going to be used instead.

When using `NO WAIT` or some other timeout value greater than 0, if the row is already locked, the lock acquisition request is going to be aborted with the following exception:

```
ORA-00054: resource busy and acquire with NOWAIT specified or timeout expired
```

The exception is meant to notify the database client that the lock could not be acquired. However, getting an exception is not always desirable, especially when implementing a job queue mechanism.

For the following example, consider that `Post` entries need to be moderated to avoid spam messages or inappropriate content.

The `Post` entity is going to use a `status` attribute which indicates if the `Post` can be safely displayed or it requires manual intervention from a site administrator.

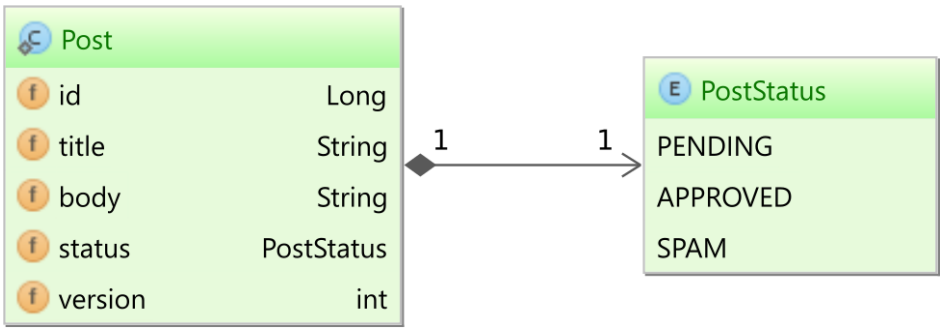

Figure 16.7: Post with PostStatus

The `Post` entities can be moderated by multiple administrators, so, in order to prevent them from approving the same entries, each administrator acquires a lock on the currently selected `Post` entities.

```
List<Post> pendingPosts = entityManager.createQuery(
    "select p " +
    "from Post p " +
    "where p.status = :status", Post.class)
.setParameter("status", PostStatus.PENDING)
.setFirstResult(5).setMaxResults(7)
.setLockMode(LockModeType.PESSIMISTIC_WRITE)
.setHint("javax.persistence.lock.timeout", 0)
.getResultList();
```

When Alice runs the aforementioned query on Oracle, Hibernate will generate the following statements:

```
SELECT * FROM (
    SELECT  row_.*, rownum rownum_ FROM (
        SELECT p.id AS id1_0_, p.body AS body2_0_, p.status AS status3_0_,
               p.title AS title4_0_, p.version AS version5_0_
        FROM   post p
        WHERE  p.status = 0
    ) row_
    WHERE   rownum <= 7
)
WHERE rownum_ > 5

SELECT id FROM post WHERE id = 5 AND version = 0 FOR UPDATE
SELECT id FROM post WHERE id = 6 AND version = 0 FOR UPDATE
```